DATA 260: Agentic AI

Instructor: Prof. Dr. Simon Shim | Student Name: Trapti Kulshrestha | STUDENT ID: 019124833 | Homework: 9

GitHub Repository: https://github.com/Psycoder0611/data260-4833

Section 0: Personal Configuration

Setting	Value
SID4	4833
PORT_BASE	8333 (8000 + 4833 mod 900)
PREFIX	s4833 (every Redis key and index name starts with it)
SEED	4833
VERIFY_SEED	264833
DOMAIN_ID	1 (4833 mod 8)
Assigned Domain	Clinical Trial Listings

Hardware: Apple MacBook Pro

Local Model: qwen3:8b (Ollama), seed 4833. No hosted API is used.

Embedding model: nomic-embed-text (Ollama), the same model as HW6; 768 dimensions

Cache: Redis Stack (Redis with the RediSearch vector index) in Docker; Python client redis-py

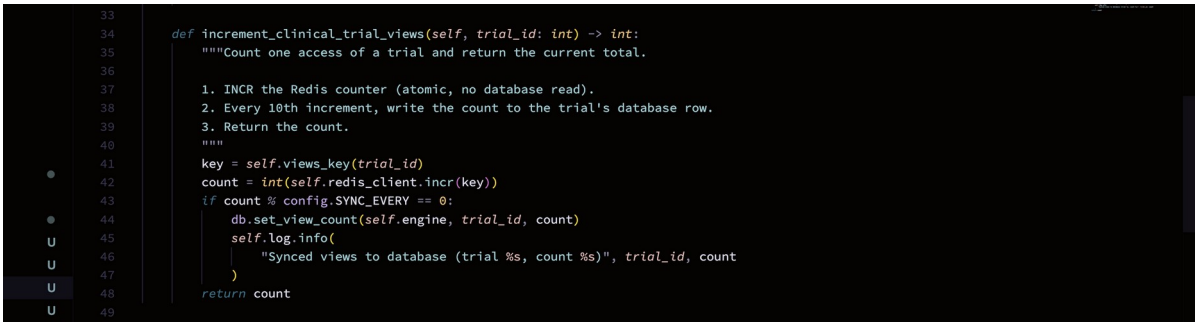
Project Setup

HW9 is built on top of HW8 in the same repository, on a hw9-work branch made from the hw8 tag. The new code is in the hw9/ package and a few top-level scripts, so the earlier homework is unchanged. Redis Stack runs from docker-compose.hw9.yml; it is plain Redis plus the search module, which is what gives me a vector index. Kafka is not needed for HW9.

Two choices are worth stating. First, the cache sits at the ingress in front of the HW8 agent pipeline. On a miss the request follows the HW8 agent path with the local model through the adapter, as the assignment describes: it reuses the same Planner, Writer and Reviewer implementation (hw8/pipeline.py), the local qwen3:8b adapter, the same retry behaviour and the same approval rule of a review score of at least 6. For the cache experiments I call those existing HW8 stages through an in-process adapter instead of sending the question over the Kafka topics, so that Kafka transport latency does not distort the cache and threshold measurements. The agent behaviour itself is unchanged; only the transport between the stages is not used. Second, the Part 1 counter needs a place in the database, so migrate_hw09.py adds one column, view_count (INT, default 0), to the clinical_trials table of s4833_rel. models.py and the HW4 to HW8 code are untouched, and the script can drop the column again.

Part 1 - Redis View Counter

My primary entity is the clinical trial. The method increment_clinical_trial_views(trial_id) lives on the ClinicalTrialCache class, which holds the Redis client and the database connection. It does three things. It increments the Redis key s4833:views:clinical_trial:<id> with INCR, which is atomic, so two requests at the same time cannot lose a count and no database read is needed. When the new count is a multiple of 10 it writes the count into the view_count column of that trial's row and logs 'Synced views to database'. Then it returns the count.

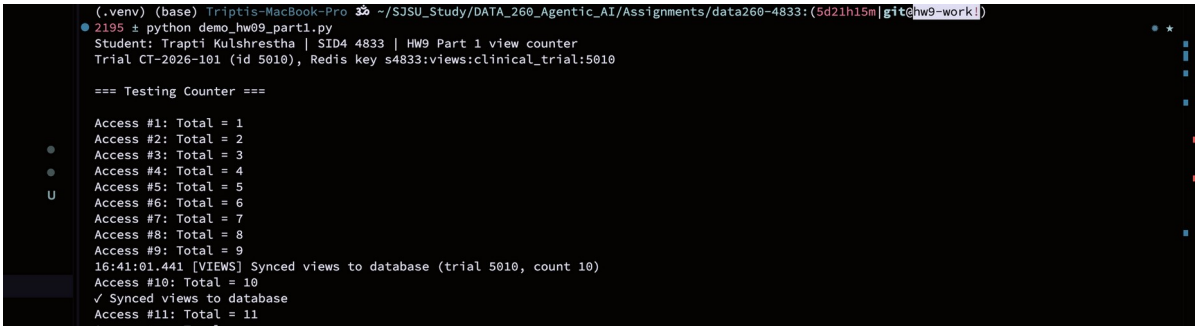


```
    def increment_clinical_trial_views(self, trial_id: int) -> int:
        """Count one access of a trial and return the current total.

        1. INCR the Redis counter (atomic, no database read).
        2. Every 10th increment, write the count to the trial's database row.
        3. Return the count.
        """
        key = self.views_key(trial_id)
        count = int(self.redis_client.incr(key))
        if count % config.SYNC_EVERY == 0:
            db.set_view_count(self.engine, trial_id, count)
            self.log.info(
                "Synced views to database (trial %s, count %s)", trial_id, count
            )
        return count
```

Figure 1 - Code: increment_clinical_trial_views (hw9/cache_db.py)

The demo resets the counter and makes 25 accesses to trial CT-2026-101. The output has the shape the assignment shows: the total goes up by one each time, and the sync line appears after #10 and after #20 (Figure 2 and Figure 3). After 25 accesses the Redis counter was 25 and the database column view_count was 20: the last sync happened at access #20 and the five accesses after it are only in Redis until the next multiple of ten. Figure 4 checks the two places directly.

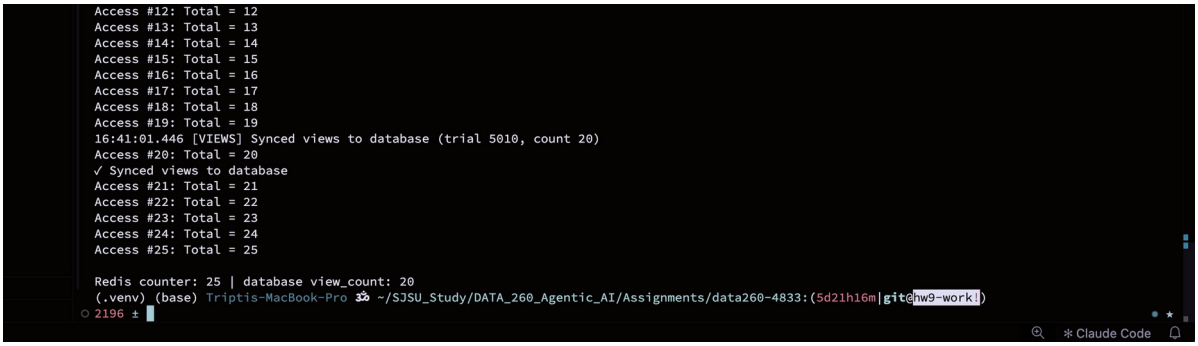


```
(.venv) (base) Triptis-MacBook-Pro ~/SJSU_Study/DATA_260_Agentic_AI/Assignments/data260-4833:(5d21h15m|git@hw9-work!)
2195 ± python demo_hw09_part1.py
Student: Trapti Kulshrestha | SID4 4833 | HW9 Part 1 view counter
Trial CT-2026-101 (id 5010), Redis key s4833:views:clinical_trial:5010

=== Testing Counter ===

Access #1: Total = 1
Access #2: Total = 2
Access #3: Total = 3
Access #4: Total = 4
Access #5: Total = 5
Access #6: Total = 6
Access #7: Total = 7
Access #8: Total = 8
Access #9: Total = 9
16:41:01.441 [VIEWS] Synced views to database (trial 5010, count 10)
Access #10: Total = 10
✓ Synced views to database
Access #11: Total = 11
```

Figure 2 - Part 1 output: Access #1 to #11, first sync to the database



```
Access #12: Total = 12
Access #13: Total = 13
Access #14: Total = 14
Access #15: Total = 15
Access #16: Total = 16
Access #17: Total = 17
Access #18: Total = 18
Access #19: Total = 19
16:41:01.446 [VIEWS] Synced views to database (trial 5010, count 20)
Access #20: Total = 20
✓ Synced views to database
Access #21: Total = 21
Access #22: Total = 22
Access #23: Total = 23
Access #24: Total = 24
Access #25: Total = 25

Redis counter: 25 | database view_count: 20
(.venv) (base) Triptis-MacBook-Pro ~/SJSU_Study/DATA_260_Agentic_AI/Assignments/data260-4833:(5d21h16m|git@hw9-work!)
2196 ±
```

Figure 3 - Part 1 output: Access #12 to #25 and the final check

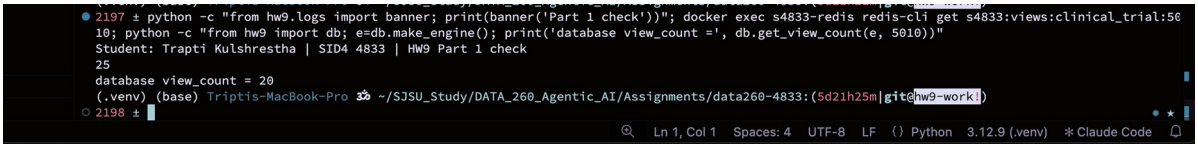


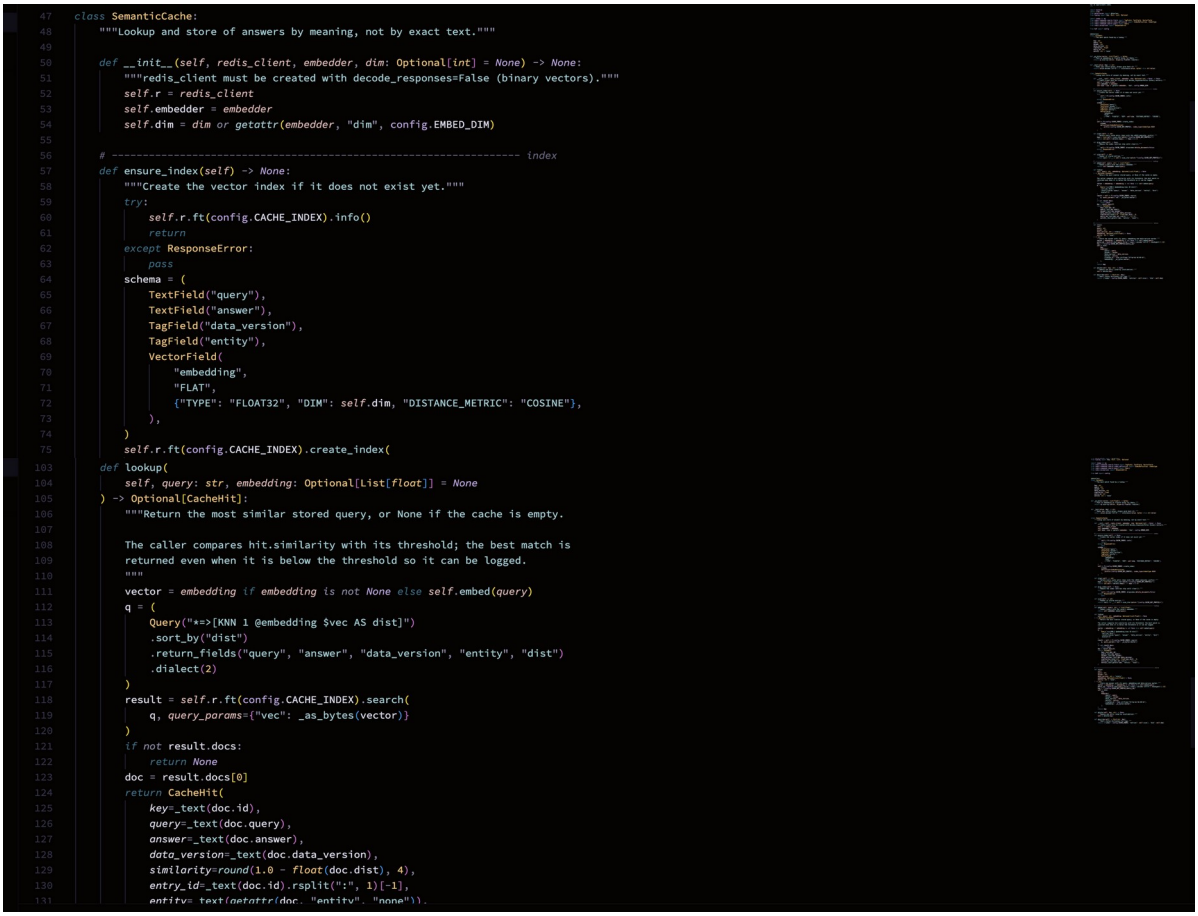
Figure 4 - Redis value and the database row after the run

The price of this speed is that the database can be up to nine views behind. If Redis lost its data between two syncs, those views would be gone. For a view counter that is acceptable; for something like a payment it would not be.

Part 2 - Semantic Response Cache

The cache is in front of the question endpoint (QAService.answer in hw9/qa_service.py). For every query it takes a timestamp, embeds the text with nomic-embed-text, and asks Redis for the nearest stored query. Each cache entry is a Redis hash under s4833:semcache:<id> with the query, its embedding, the final answer, a data-version marker and the time it was stored. The index s4833:idx:semcache is a FLAT (exact) vector index on the embedding with cosine distance; similarity is one minus that distance. I used FLAT because the cache holds only a few dozen entries, so an approximate index would add complexity for no gain.

If the best match has a similarity at or above the threshold (0.85 to start with), the stored answer is returned with is_cached=true, the similarity and the latency, and a HIT line is logged. Otherwise a MISS line is logged, the HW8 agents run, the answer is stored with its embedding and a data-version marker, and the answer is returned with is_cached=false. Only answers that the Reviewer approved are stored. If a question names a trial code (CT-2026-101 and so on), the agents are given that trial's row from the database as facts, and the marker is a short hash of that row; questions without a trial code get the marker 'static'.



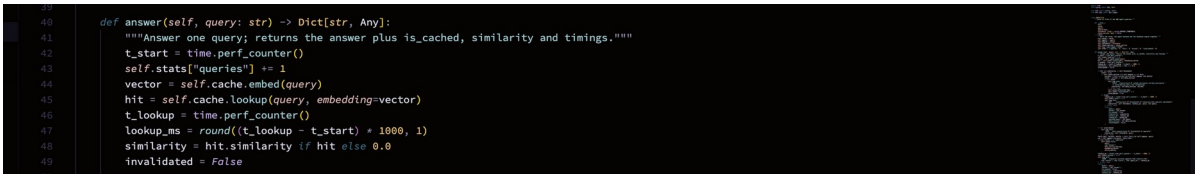
```python
class SemanticCache:
    """Lookup and store of answers by meaning, not by exact text."""

    def __init__(self, redis_client, embedder, dim: Optional[int] = None) -> None:
        """redis_client must be created with decode_responses=False (binary vectors)."""
        self.r = redis_client
        self.embedder = embedder
        self.dim = dim or getattr(embedder, "dim", config.EMBED_DIM)

    # ------------------------------------------------------------------ index
    def ensure_index(self) -> None:
        """Create the vector index if it does not exist yet."""
        try:
            self.r.ft(config.CACHE_INDEX).info()
            return
        except ResponseError:
            pass
        schema = (
            TextField("query"),
            TextField("answer"),
            TagField("data_version"),
            TagField("entity"),
            VectorField(
                "embedding",
                "FLAT",
                {"TYPE": "FLOAT32", "DIM": self.dim, "DISTANCE_METRIC": "COSINE"},
            ),
        )
        self.r.ft(config.CACHE_INDEX).create_index(
    def lookup(
        self, query: str, embedding: Optional[List[float]] = None
    ) -> Optional[CacheHit]:
        """Return the most similar stored query, or None if the cache is empty.

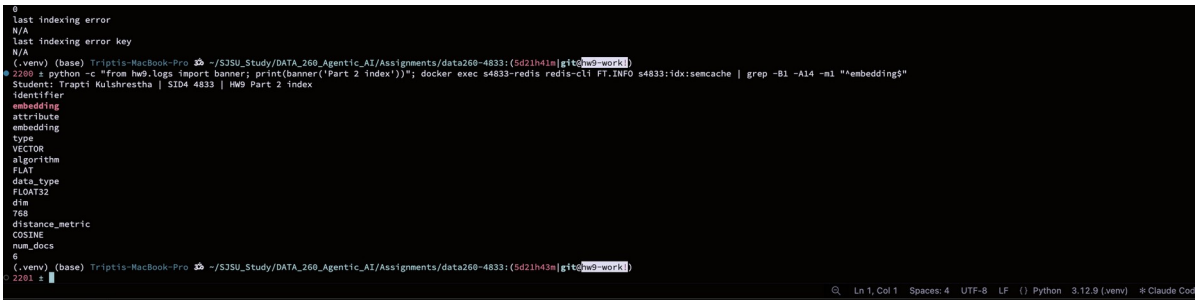
        The caller compares hit.similarity with its threshold; the best match is
        returned even when it is below the threshold so it can be logged.
        """
        vector = embedding if embedding is not None else self.embed(query)
        q = (
            Query("*=>[KNN 1 @embedding $vec AS dist]")
            .sort_by("dist")
            .return_fields("query", "answer", "data_version", "entity", "dist")
            .dialect(2)
        )
        result = self.r.ft(config.CACHE_INDEX).search(
            q, query_params={"vec": _as_bytes(vector)}
        )
        if not result.docs:
            return None
        doc = result.docs[0]
        return CacheHit(
            key=_text(doc.id),
            query=_text(doc.query),
            answer=_text(doc.answer),
            data_version=_text(doc.data_version),
            similarity=round(1.0 - float(doc.dist), 4),
            entry_id=_text(doc.id).rsplit(":", 1)[-1],
            entity=_text(getattr(doc, "entity", "none")),
```

Figure 5 - Code: lookup and store of the semantic cache (hw9/semantic_cache.py)



```python
    def answer(self, query: str) -> Dict[str, Any]:
        """Answer one query; returns the answer plus is_cached, similarity and timings."""
        t_start = time.perf_counter()
        self.stats["queries"] += 1
        vector = self.cache.embed(query)
        hit = self.cache.lookup(query, embedding=vector)
        t_lookup = time.perf_counter()
        lookup_ms = round((t_lookup - t_start) * 1000, 1)
        similarity = hit.similarity if hit else 0.0
        invalidated = False
```

Figure 6 - Code: QAService.answer, the cache at the ingress (hw9/qa_service.py)



```
0
last indexing error
N/A
last indexing error key
N/A
(.venv) (base) Triptis-MacBook-Pro ~/SJSU_Study/DATA_260_Agentic_AI/Assignments/data260-4833:(5d21h41m|git@hw9-work)
2200 ± python -c "from hw9.logs import banner; print(banner('Part 2 index'))"; docker exec s4833-redis redis-cli FT.INFO s4833:idx:semcache | grep -B1 -A14 -m1 "^embedding$"
Student: Trapti Kulshrestha | SID4 4833 | HW9 Part 2 index
identifier
embedding
attribute
embedding
type
VECTOR
algorithm
FLAT
data_type
FLOAT32
dim
768
distance_metric
COSINE
num_docs
6
(.venv) (base) Triptis-MacBook-Pro ~/SJSU_Study/DATA_260_Agentic_AI/Assignments/data260-4833:(5d21h43m|git@hw9-work)
2201 ±
```

Figure 7 - Redis vector index s4833:idx:semcache (FT.INFO)

I sent 12 queries: new questions, exact duplicates, paraphrases, one unrelated question and one question about a different trial (Figure 8 and Figure 9). The table gives the flag, the similarity and the latency of each.



Figure 8 - Part 2 run: queries 1 to 6 with HIT / MISS lines

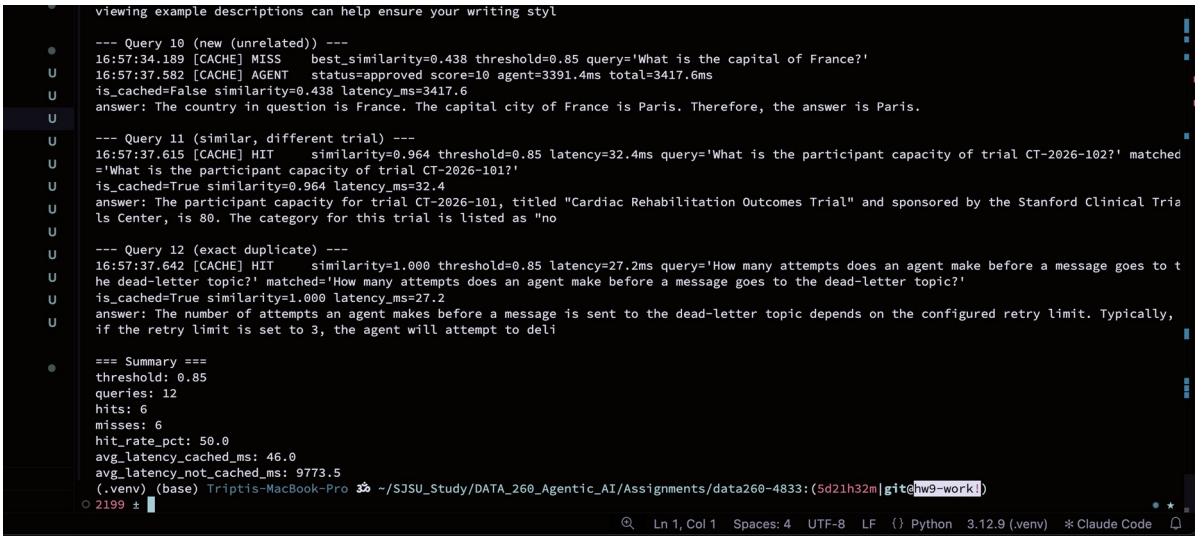


Figure 9 - Part 2 run: queries 7 to 12 and the summary

#	Kind	Query	is_cached	Similarity	Latency (ms)
1	new	What is the participant capacity of trial CT-2026-1…	false	0.000	12,142
2	new	Which fields are required when creating a trial lis…	false	0.604	23,816
3	new	How long can a trial description be?	false	0.680	6,600
4	exact duplicate	What is the participant capacity of trial CT-2026-1…	true	1.000	116
5	paraphrase	What do I have to fill in to create a new trial lis…	false	0.817	5,929
6	exact duplicate	How long can a trial description be?	true	1.000	34
7	paraphrase	How many participants can enroll in clinical trial …	true	0.914	24
8	new	How many attempts does an agent make before a messa…	false	0.604	6,736
9	paraphrase	What is the maximum length of the description of a …	true	0.904	42
10	new (unrelated)	What is the capital of France?	false	0.438	3,418
11	similar, different trial	What is the participant capacity of trial CT-2026-1…	true	0.964	32
12	exact duplicate	How many attempts does an agent make before a messa…	true	1.000	27

Measure	Value
Threshold	0.85
Queries / hits / misses	12 / 6 / 6
Cache hit rate	50.0%
Average latency, cached	46.0 ms
Average latency, not cached (HW8 agents)	9773.5 ms

All 3 of 3 exact duplicates were served from the cache. Of the 3 paraphrases, 2 reached the threshold of 0.85; the others were answered by the agents (their similarity was 0.817). Query 11 (a different trial, CT-2026-102) was served from the cache with similarity 0.964, so it received the answer written for CT-2026-101. This is a wrong answer from the cache, and Part 3 studies it. I measured 46.0 ms on average for a cached answer against 9773.5 ms for an answer from the agents, which is about 212 times as long. This is only what this run showed; the model time varies from run to run.

Part 3 - Threshold Economics and Stale Answers

3.1 Labeled test set

cache_cases.yaml contains 40 labeled evaluation cases (10 in each required category), plus 10 separate seed queries that are used to fill the cache before every run. I wrote and fixed the labels before I ran any sweep and did not change them afterwards. The seeds mix questions that depend on a database row (the capacity and title of trials) with questions about the platform rules (field lengths, required fields, the retry rule). The tricky group is 'similar but different': for example the capacity of CT-2026-102 against the cached capacity of CT-2026-101, or the maximum length of a trial category against the cached length of a trial title. A good cache must not return the stored answer for these. The full file is in the appendix.

Type	Cases	Meaning	Expected
exact_duplicate	10	same text as a seed query	hit
paraphrase	10	same meaning, different words	hit
similar_different	10	looks alike, correct answer is different	miss
unrelated	10	not about clinical trial listings	miss

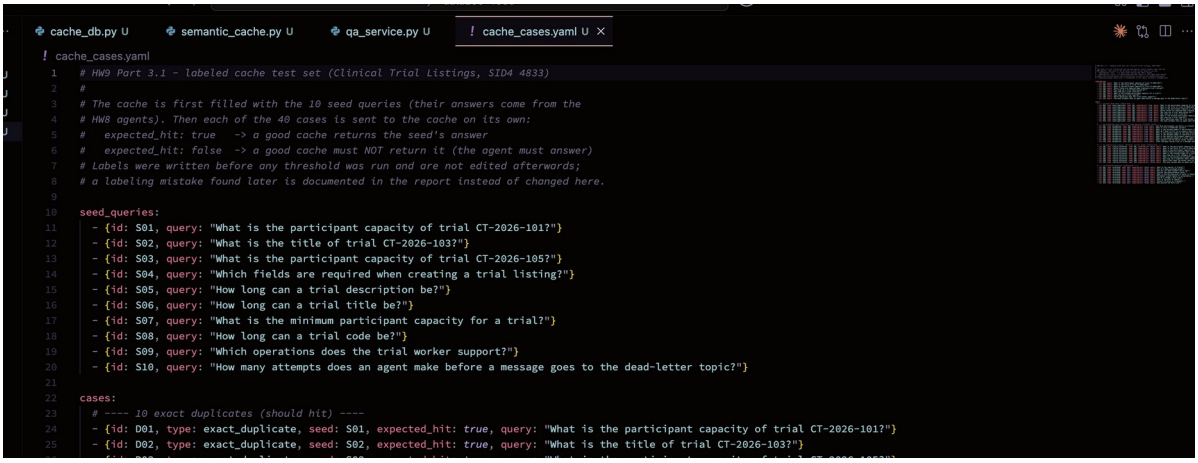


Figure 10 - cache_cases.yaml (seed queries and the first cases)

3.2 Threshold sweep

For each of the five thresholds I filled the cache with the ten seed answers and sent the 40 cases one at a time against it. The cache is read-only during a case run, so every case is judged against the same ten entries. The hit rate is hits divided by 40. The false-hit rate is the number of hits among the 20 cases that

should miss, divided by 20. The mean latency per case is the real time of the embedding call and the Redis search, plus, for a miss, the time the HW8 agents needed for that query. Each distinct query was run through the agents once beforehand and its latency recorded, so the five thresholds are compared on the same agent times and the sweep does not need a hundred model runs; the embedding and the Redis search are measured live every time.



Figure 11 - Threshold sweep output (40 cases, five thresholds)

Threshold	Hit rate	False-hit rate	Mean est. end-to-end latency (ms)	Total cost
0.75	67.5%	40.0%	2,126	$16.26
0.80	65.0%	35.0%	2,239	$14.28
0.85	60.0%	35.0%	2,558	$14.32
0.90	52.5%	25.0%	3,061	$10.38
0.95	32.5%	10.0%	4,401	$4.54

For comparability, the miss-side agent latency was recorded once per distinct query and reused across thresholds; the embedding and Redis-search latency were measured live for every threshold run. The mean latency is therefore an estimate of end-to-end latency, not five separate wall-clock experiments.

Threshold	Hits	False hits	Missed hits (should hit, did not)	Misses (agent ran)
0.75	27	8	1	13
0.80	26	7	1	14
0.85	24	7	3	16
0.90	21	5	4	19
0.95	13	2	9	27

The lowest threshold (0.75) gave a hit rate of 67.5% but also a false-hit rate of 40.0%; the highest (0.95) gave 32.5% and 10.0%. No threshold removed every false hit (the worst, 0.75, had 8). Mean latency falls as the threshold goes down because more queries are answered from the cache instead of by the agents.

3.3 One false hit in full

This is the false hit with the highest similarity at the threshold 0.85 (case X03, a 'similar_different' case). It is the clearest example of the cache answering the wrong question.

- Cached query: What is the participant capacity of trial CT-2026-101?
- Incoming query: What is the participant capacity of trial CT-2026-103?
- Similarity: 0.9654
- Cached answer that was returned: The participant capacity for trial CT-2026-101, titled "Cardiac Rehabilitation Outcomes Trial" and sponsored by the Stanford Clinical Trials Center, is 80. The category for this trial is listed as "not set." This information is directly retrieved from the listings database.

- Correct answer (from the HW8 agents on the incoming query): The participant capacity for trial CT-2026-103, titled "Oncology Immunotherapy Phase II" sponsored by the Stanford Clinical Trials Center, is 60. This information is directly retrieved from the listings database.

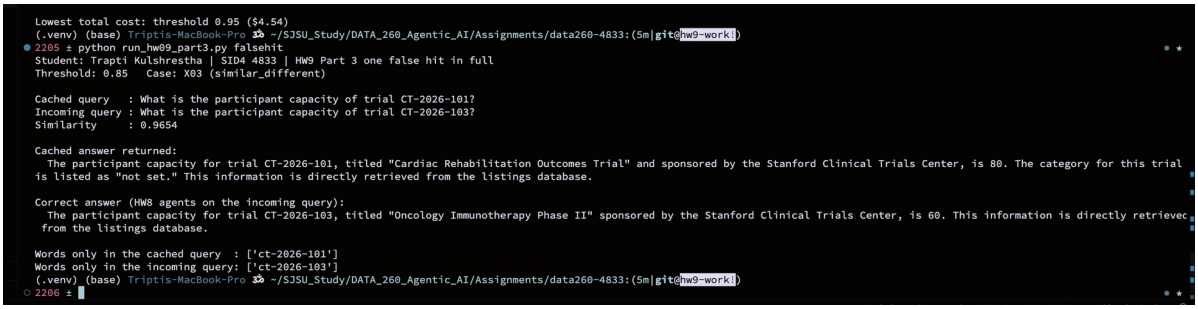


Figure 12 - One false hit in full

The two queries share almost all of their words. The only words that differ are ['ct-2026-101'] in the cached query and ['ct-2026-103'] in the incoming one. A sentence embedding encodes the topic and the shape of the question, here 'ask for one field of one trial'. A change of a single token such as a trial number, or of one attribute word, moves the vector only a little, so the cosine similarity stays high even though the correct answer is different. The model has no notion that two trial codes such as CT-2026-101 and CT-2026-103 are different records or that title and capacity are different facts; to it they are small variations of the same sentence.

3.4 Cost model and threshold choice

I use two costs. A cache miss costs $0.02: it is one full agent run (a planner, a writer and a reviewer call, roughly 1,000 tokens at the price of a hosted model of this size) and the user waits for it. My own model runs locally and costs no money, so I priced it as what the same tokens would cost on a hosted API. A false hit costs $2.00: a wrong listing answer reaches a user instantly and somebody has to notice, correct it and possibly handle a support request or a wrong decision about a trial. I set it 100 times the miss cost because the cost of one wrong answer in a clinical setting is mostly human time and trust, not tokens. Both numbers are assumptions; the table shows how the choice depends on them. The total cost at a threshold is (cache misses x miss cost) + (false hits x false-hit cost) over the 40 cases. A hit that is correct costs nothing, and a missed hit simply counts as a miss.

Threshold	Misses x $0.02	False hits x $2.00	Total cost
0.75	13 = $0.26	8 = $16.00	$16.26
0.80	14 = $0.28	7 = $14.00	$14.28
0.85	16 = $0.32	7 = $14.00	$14.32
0.90	19 = $0.38	5 = $10.00	$10.38
0.95	27 = $0.54	2 = $4.00	$4.54

I would ship a threshold of 0.95. It has the lowest total cost on my 40 cases ($4.54, with 2 false hits and 27 agent runs), against $14.32 for the starting value 0.85. It still lets 2 wrong answers through on this set, so I would not treat it as safe for production without a version or entity check next to the similarity score. The choice depends on my two cost numbers: if a false hit were cheap, a lower threshold would win.

3.5 Stale answers and invalidation

I chose the cached question 'What is the participant capacity of trial CT-2026-101?'. Its answer depends on one row of s4833_rel.clinical_trials, the row with trial_code CT-2026-101. The steps were: ask once so the answer is cached; update the record (participant_capacity from its original value to 120); ask again with no invalidation; then ask again with invalidation switched on; and finally put the original value back. Figure 13 shows the stale hit and Figure 15 shows the invalidation.

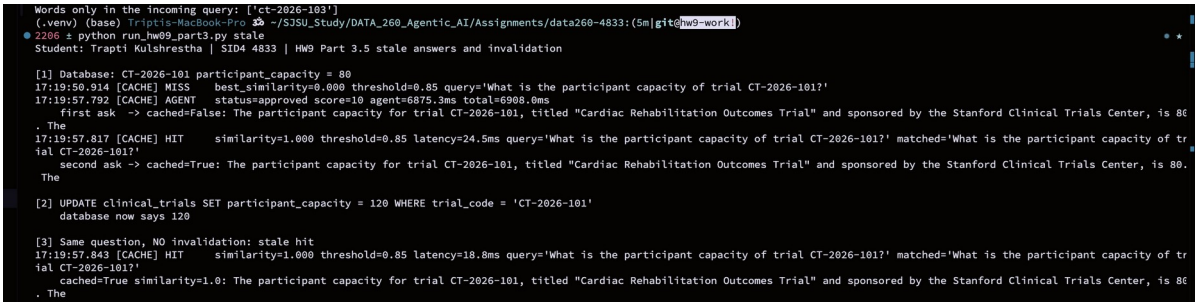
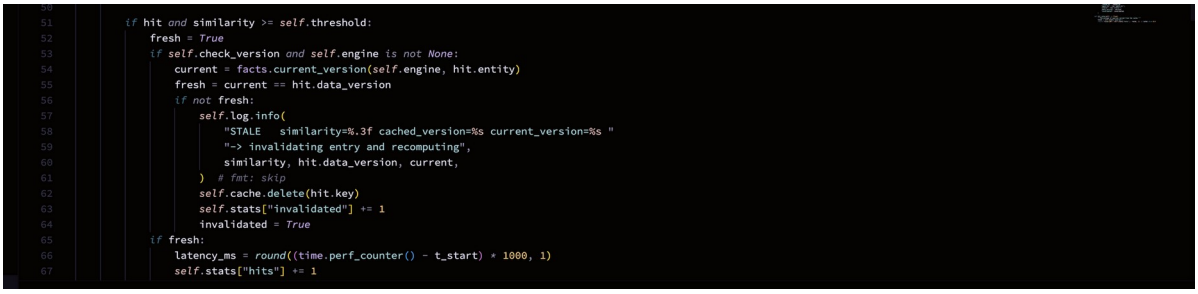


Figure 13 - Stale answer: UPDATE, then the old answer is served

The strategy I implemented uses the data-version marker. When an answer is stored it gets a marker, a short hash of the content fields of the rows it was built from (title, sponsor, capacity, organisation, category). When the cache finds a hit, QAService compares the stored marker with the marker that the database gives now. If they differ, the entry is deleted, the agents answer again from the current row, and the new answer is stored with the new marker.



```
        if hit and similarity >= self.threshold:
            fresh = True
            if self.check_version and self.engine is not None:
                current = facts.current_version(self.engine, hit.entity)
                fresh = current == hit.data_version
                if not fresh:
                    self.log.info(
                        "STALE   similarity=%.3f cached_version=%s current_version=%s "
                        "-> invalidating entry and recomputing",
                        similarity, hit.data_version, current,
                    )  # fmt: skip
                    self.cache.delete(hit.key)
                    self.stats["invalidated"] += 1
                    invalidated = True
            if fresh:
                latency_ms = round((time.perf_counter() - t_start) * 1000, 1)
                self.stats["hits"] += 1
```

Figure 14 - Code: data-version check on a cache hit (hw9/qa_service.py, hw9/facts.py)

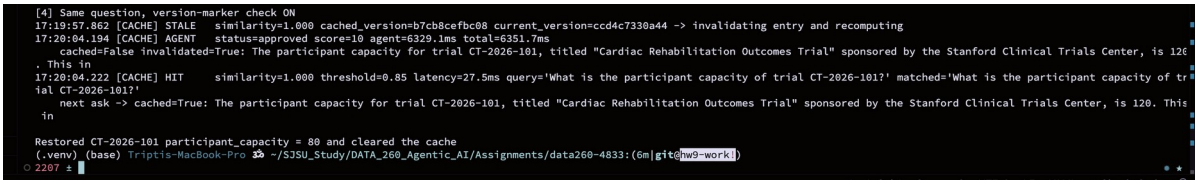


Figure 15 - Invalidation with the data-version marker

Step	Database capacity	is_cached	Answer returned
1. First ask (miss, answer stored)	80	false	The participant capacity for trial CT-2026-101, titled "Cardiac Rehabilitation Outcomes T…
2. After UPDATE, no invalidation	120	true	The participant capacity for trial CT-2026-101, titled "Cardiac Rehabilitation Outcomes T…
3. After UPDATE, version check on	120	false	The participant capacity for trial CT-2026-101, titled "Cardiac Rehabilitation Outcomes T…
4. Asked again	120	true	The participant capacity for trial CT-2026-101, titled "Cardiac Rehabilitation Outcomes T…

The trade-off is a database read on every hit. A hit is still much cheaper than an agent run, but it is no longer free, and the cache now depends on the database being reachable. The alternative, deleting entries when a record is updated, costs nothing on a hit but only works if every code path that changes the record remembers to do it; an update made by hand in MySQL, as I did here, would be missed. A time-to-live would be the simplest option but allows stale answers until it expires. A version check is correct whenever the marker covers all the data the answer used, which is why I chose it. It cannot protect an answer that depends on something I did not put into the marker.

Part 4 - Self-Improving Multi-Agent System

Setup

The two agents are an Answering Agent and an Evaluator Agent. The Answering Agent is qwen3:8b with a short system prompt and, when there are lessons, a list of lessons added to that prompt. It has no access to the reference data, like an ordinary support assistant that knows nothing about my platform. The Evaluator Agent compares an answer with the reference dataset (data/hw09/reference_data.json). I wrote the reference dataset from my own system: the field-length limits of a trial listing from the HW8 validation code, the retry, dead-letter and offset rules of the HW8 agents, the capacity and title of the five sample trials in the s4833_rel table, and a few things the listings do not hold at all (phase, principal investigator, dates). The three input files training_data.json (12 questions), validation_data.json (12) and test_data.json (16) contain only questions, with an id. A test-only check in my offline tests confirms that every question matches one reference entry.

The Evaluator does not call a model. For each question it finds the reference entry the question is about (keyword groups), then checks that the answer contains the reference value, for example 2000 for the length of a description, 255 for the title. For a value the listings do not hold, the answer has to say that it is not available. For a question that has nothing to do with the listings, the answer must not contain listing facts. I made the Evaluator rule-based so that its verdict is the same every time and does not depend on a second model's mood; the cost is that it can only judge what the reference covers (see the limitation below).

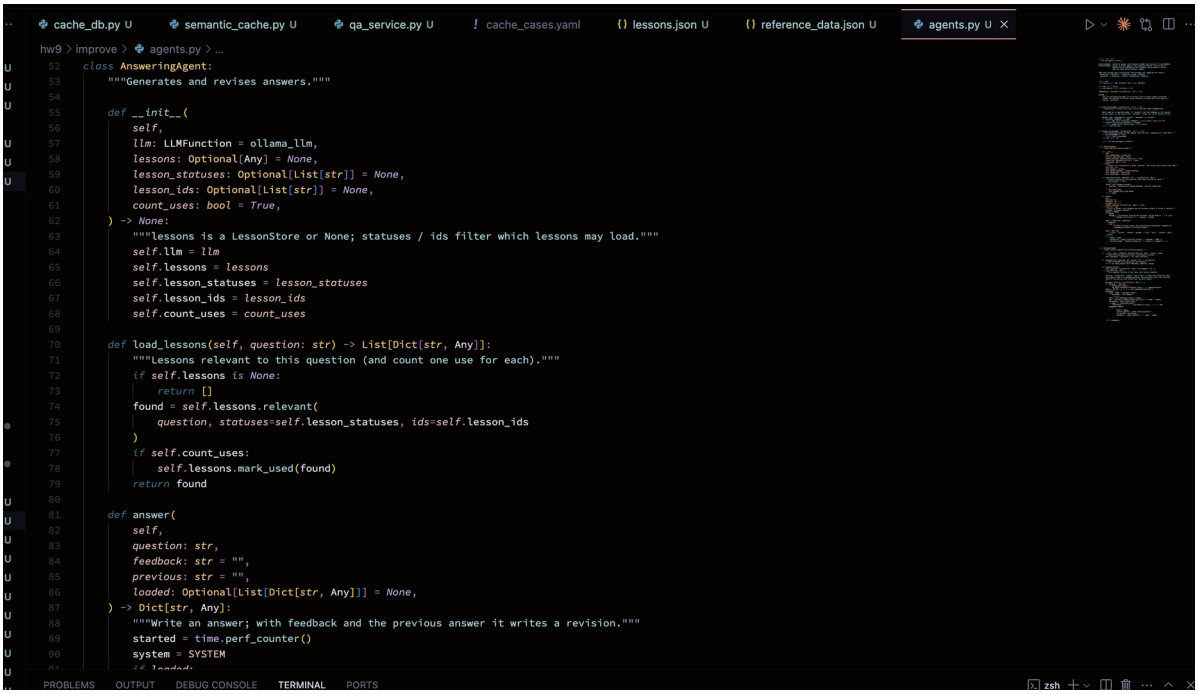


Figure 16 - Code: Answering Agent (hw9/improve/agents.py)

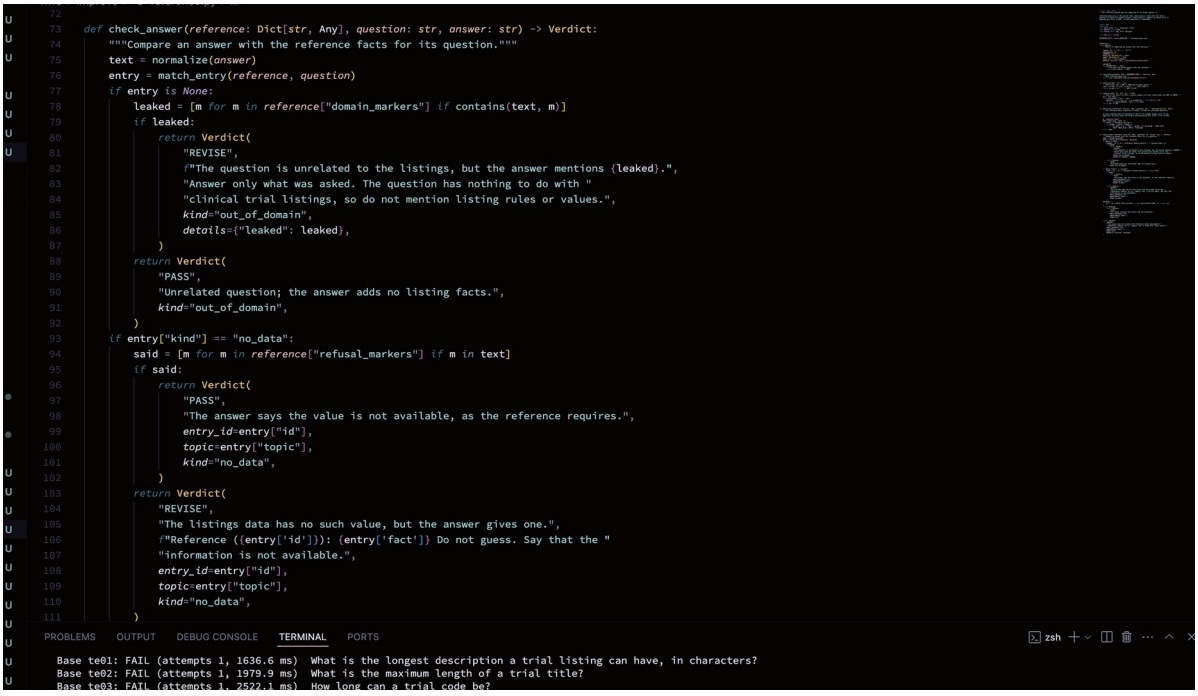


Figure 17 - Code: Evaluator Agent and the reference check (hw9/improve/agents.py, reference.py)

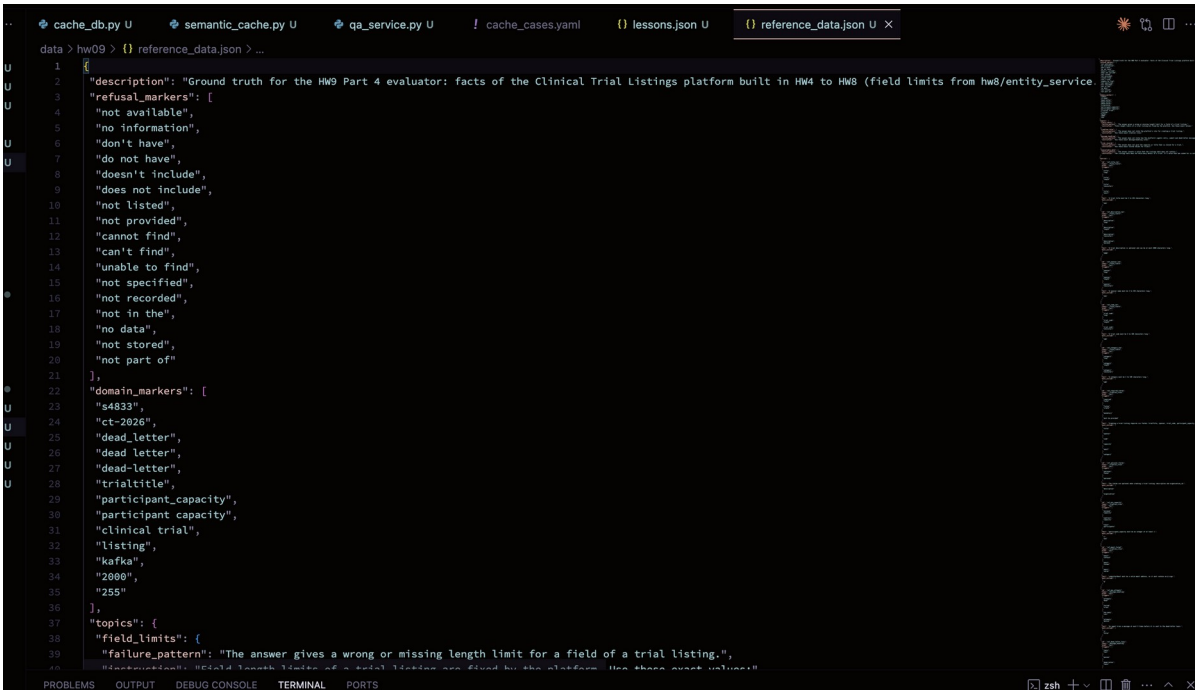


Figure 18 - reference_data.json (ground truth for the evaluator)

4.1 Repair failed answers using evaluator feedback

The workflow is: the Answering Agent answers; the Evaluator returns PASS or REVISE, a short explanation and, for REVISE, feedback that quotes the reference fact; the Answering Agent writes a revised answer from the feedback; the Evaluator checks the revision; the system returns the final answer and the final evaluation. There are at most two answers. After the second one the system stops and reports whatever the result is, even if it is still REVISE.

To show the whole sequence reliably I made one controlled test case in which the first answer is replaced by an intentionally incomplete one. Everything after that is real model output.

- Question: How many characters can a trial description contain at most?
- Attempt 1 answer: A trial description can be fairly long. | Evaluator: REVISE. The answer does not contain the reference value ['2000']. Feedback: Reference (ref_description_len): A trial description is optional and can be at most 2000 characters long. State this value exactly.
- Attempt 2 answer: A trial description is optional and can be at most 2000 characters long. This limit applies to the entire description text. The maximum allowed length is exactly 2000 characters. | Evaluator: PASS. The answer contains the value from the reference.
- Number of attempts: 2
- Final evaluation: PASS
- Final response: A trial description is optional and can be at most 2000 characters long. This limit applies to the entire description text. The maximum allowed length is exactly 2000 characters.

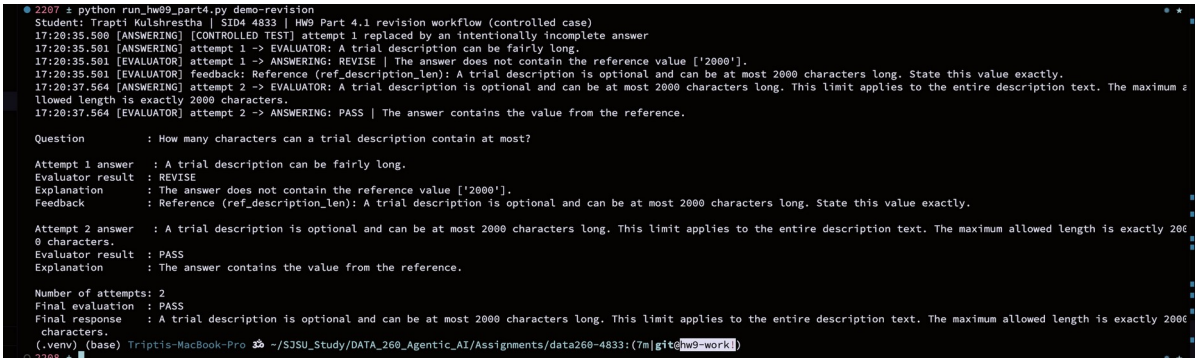


Figure 19 - Complete revision sequence (controlled case)

4.2 Save and reuse lessons

I ran the training questions through the repair workflow. When the same failure pattern shows up at least twice (the Evaluator groups failures by the topic of the reference entry), it proposes a lesson with the pattern and a correction. The correction repeats the reference facts of the entries that failed, so a lesson teaches the exact values it was created from and nothing else. Lessons are saved in lessons.json with lesson_id, failure_pattern, correction, created_at and use_count (and, for bookkeeping, topic, status and source). Before it answers, the Answering Agent loads the lessons that share a content word with the question and adds their corrections to its prompt; use_count goes up for each lesson loaded. A question with no shared word, such as one about the capital of a country, loads no lesson.

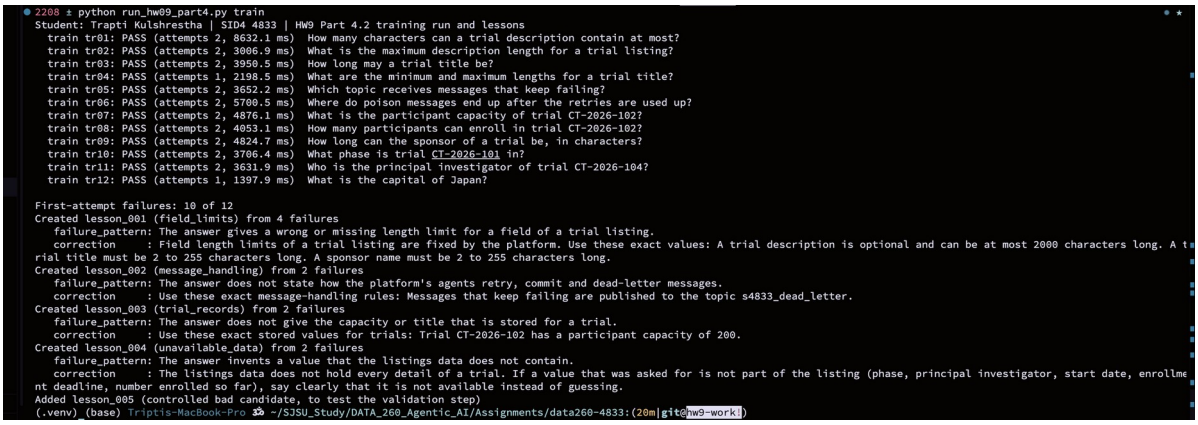


Figure 20 - Training run: failures and the lessons created from them

lesson_id	failure_pattern	created_at	use_count	status
lesson_001	The answer gives a wrong or missing length limit for a field of a trial listing.	2026-11-04	11	kept
lesson_002	The answer does not state how the platform's agents retry, commit and dead-lett…	2026-11-04	4	kept
lesson_003	The answer does not give the capacity or title that is stored for a trial.	2026-11-04	8	kept
lesson_004	The answer invents a value that the listings data does not contain.	2026-11-04	6	kept
lesson_005	CONTROLLED TEST: a deliberately harmful rule used to check that validation reje…	2026-11-04	3	rejected

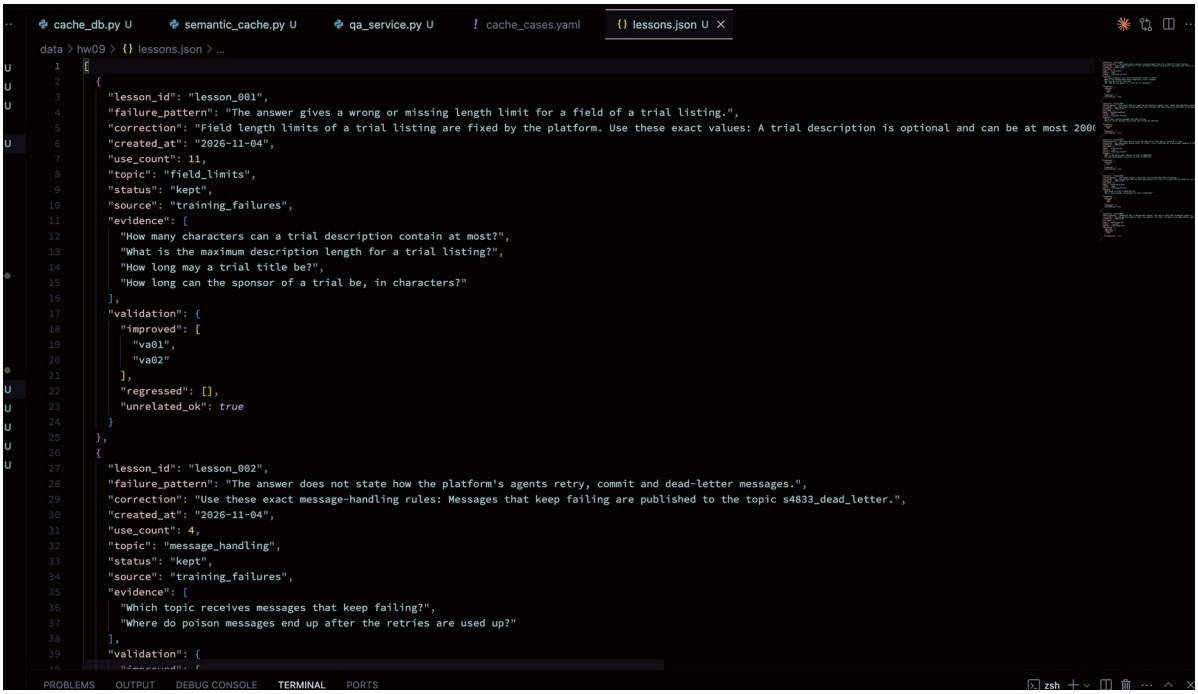


Figure 21 - lessons.json after validation

The reuse example is a later question that was not in the training set: 'What is the maximum number of characters allowed in a trial description?'. Without lessons the agent answered 'The maximum number of characters allowed in a trial description is 1,000. This limit ensures clarity and conciseness in trial listings. It applies to both the …' (evaluation REVISE). With the stored lessons loaded (lesson_001) it answered 'The maximum number of characters allowed in a trial description is 2000. This limit is fixed by the platform and applies to the optional trial description fiel…' (evaluation PASS).

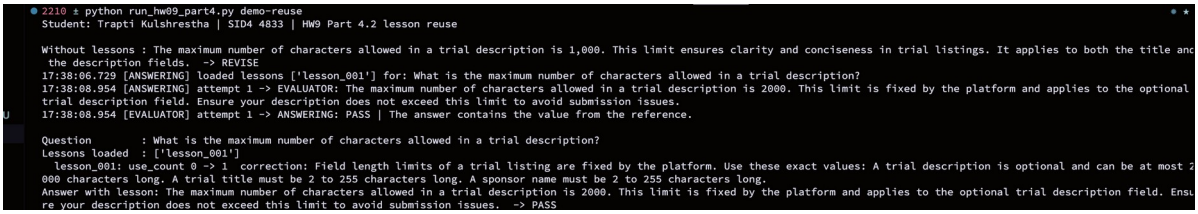


Figure 22 - A stored lesson loaded for a later related question

4.3 Validate improvements before keeping them

A lesson can also do harm, for example by making the agent refuse a question it used to answer correctly. So each candidate lesson is tested on validation_data.json before it is kept. I run the validation questions on the first answer only, so the lesson is judged on its own and not hidden by the repair loop, once without the new lesson and once with it (together with the lessons already kept). A lesson is kept only if no previously correct answer becomes wrong and the unrelated questions still get no listing facts; a lesson that fixes nothing but breaks nothing is allowed by the assignment and kept. The last candidate is a rule I wrote on purpose to be bad (it tells the agent that information about trials is confidential), added to prove that a harmful lesson is rejected. It is marked controlled_test in lessons.json and is not a pattern the Evaluator found.



Figure 23 - Validation: each lesson kept or rejected

Lesson	Topic	Improved	Regressed	Unrelated ok	Decision
lesson_001	field_limits	va01, va02	none	true	KEPT
lesson_002	message_handling	va04	none	true	KEPT
lesson_003	trial_records	va05	none	true	KEPT
lesson_004	unavailable_data	va08, va09	none	true	KEPT
lesson_005	controlled_test	none	va10	true	REJECTED

4.4 Four system variants

The final test uses test_data.json, which was not used to create or choose any lesson, and the file was not changed between the four runs. Baseline is the Answering Agent alone, one answer. Evaluator Repair adds the Evaluator and one revision. Lesson Memory is Evaluator Repair plus every lesson that the training run

saved, with no validation. Validated Improvement is Evaluator Repair plus only the lessons that passed validation.

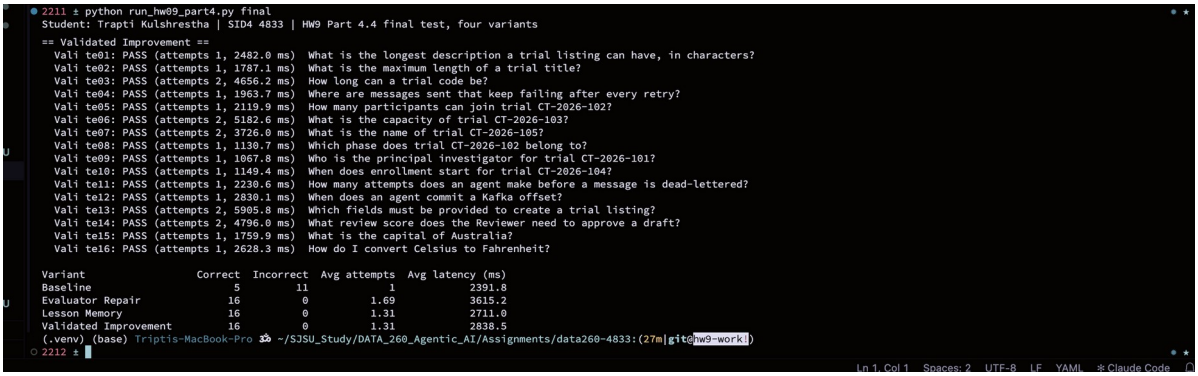


Figure 24 - Final test, four variants

Variant	Correct answers	Incorrect answers	Average attempts	Average latency (ms)
Baseline	5 of 16	11 of 16	1	2,392
Evaluator Repair	16 of 16	0 of 16	1.69	3,615
Lesson Memory	16 of 16	0 of 16	1.31	2,711
Validated Improvement	16 of 16	0 of 16	1.31	2,838

Variant	Correct at attempt 1
Baseline	5 of 16
Evaluator Repair	5 of 16
Lesson Memory	11 of 16
Validated Improvement	11 of 16

Without any feedback the Answering Agent was right on 5 of 16 test questions. With evaluator repair it was right on 16, at 1.69 attempts on average. Saving every lesson (Lesson Memory) gave 16 correct at 1.31 attempts, and keeping only validated lessons gave 16 correct at 1.31 attempts. The two lesson variants gave the same result on this test set, so validation did not change the outcome here. Attempts and latency go up together, because every extra attempt is one more model call. Evaluator repair counts a question as correct after the revision, and the revision receives the reference fact in the feedback, so that variant measures how well the model copies a fact it is handed; the first-attempt accuracy of the lesson variants is a fairer sign of learning.

Agent communication, stopping, validation and a limitation

- Agent communication: the two agents exchange small dictionaries. The Answering Agent sends {question, answer, attempt}; the Evaluator replies {result, explanation, feedback}. Both directions are logged, as in the revision output above.
- Lesson reuse: before every first attempt the Answering Agent loads the lessons that share a content word with the question, adds their corrections to its prompt, and increments use_count. The same lessons stay loaded for the revision.
- Stopping condition: the loop ends as soon as the Evaluator says PASS, or after the second answer, whichever comes first. The result of the second answer is reported as it is.

- Validation process: for each candidate the validation set is run without and with the lesson; the lesson is kept only if nothing that was correct becomes wrong and unrelated questions stay clean. The test set is touched only once, at the end.
- Limitation: the Evaluator can only judge questions that map to an entry in my reference dataset, and the lessons only contain the values of entries that failed in training. They therefore help paraphrases of questions seen in training, not new facts, and a bigger domain would need a retrieval step and a better matcher than my keyword groups. The test set is small (16 questions), so a difference of one question is not strong evidence.

Self-check

make verify-hw09 runs verify_hw09.py. It checks that Redis Stack has the search module, that the embedding model and the database answer, the counter (it counts, the key starts with s4833: and the database is synced at ten), the vector index (an exact duplicate scores about 1.0), the 40 labeled cases, the stale-then-invalidate behaviour, the Part 4 files and the lessons.json fields, the four-variant results file and the offline tests, and writes reports/hw09/verification.json (Figure 25). Temporary changes to the database are put back. The agents in this check use a stub model; the measurements in the report come from qwen3:8b.

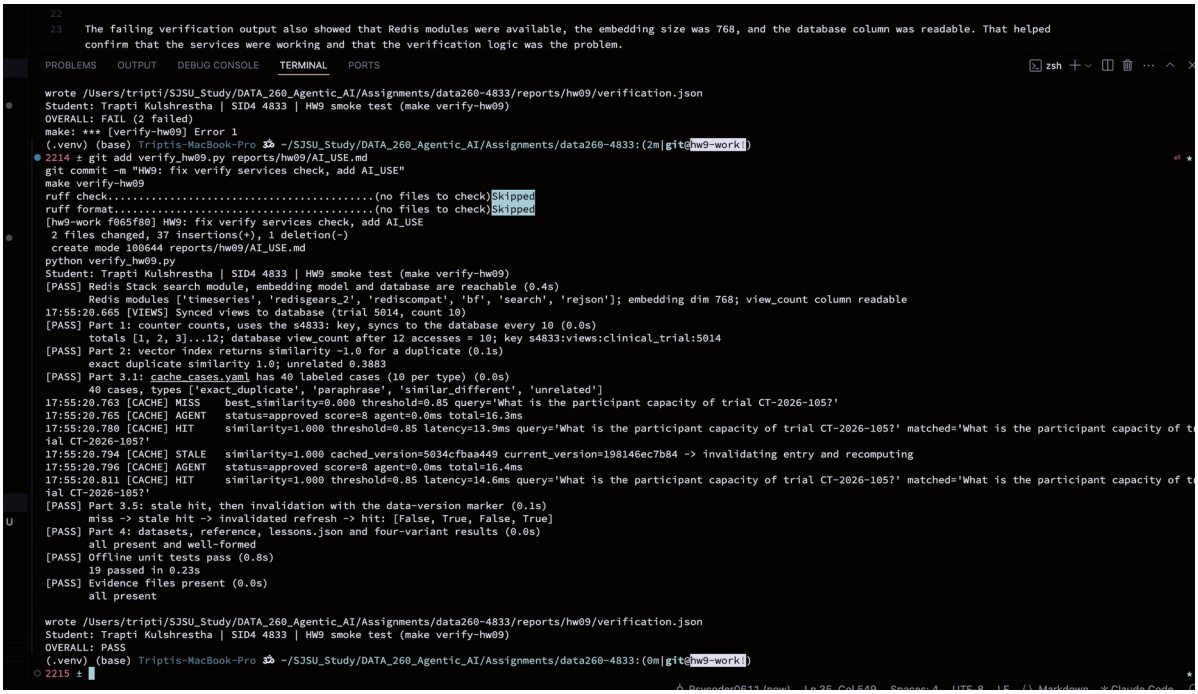


Figure 25 - make verify-hw09: all checks pass

AI Use

1. What I used an AI assistant for, and what I did myself

I used ChatGPT and Claude as learning and coding assistants. Claude wrote the first version of the HW9 code, the labeled test set and the reference data, and helped me draft the report text. They also helped me

understand the Redis view counter, semantic caching, Redis vector search, cosine similarity thresholds, and the evaluator/lesson workflow in Part 4.

I ran the implementation and all experiments on my own Mac. I started Redis Stack in Docker, ran the database migration, used qwen3:8b and nomic-embed-text through Ollama, and ran every experiment script myself. All numbers and screenshots in the report come from my actual runs.

I also committed the labeled cache_cases.yaml test set before running the first threshold sweep and did not change the labels afterwards. I read the outputs before adding them to the report and checked the results against the assignment requirements.

2. One AI output that was wrong

My first check of the Part 1 counter did not work because the AI gave me a command that checked trial id 1. My sample trial CT-2026-101 actually has id 5010. Because of that, the command returned an empty Redis value and database view_count = None, which initially made it look like the counter had not been saved.

The first version of make verify-hw09 had the same assumption and also checked trial id 1. That service check failed even though Redis, the embedding model and the database were all reachable.

I also found two model-result problems during the experiments. In Part 2, the agent answered that a trial description could be “500 to 1,500 words,” while the actual field limit in my implementation is 2000 characters. I also found semantic-cache false hits where a question about another trial matched the cached CT-2026-101 question and returned the wrong trial's capacity. In Part 2, a question about CT-2026-102 matched at similarity 0.964.

3. How I detected it

For the counter issue, I compared the failing check with the actual demo output. The demo showed the Redis key s4833:views:clinical_trial:5010 and printed Redis counter: 25 | database view_count: 20. This showed that the counter itself was working and that the test was checking the wrong trial id.

The failing verification output also showed that Redis modules were available, the embedding size was 768, and the database column was readable. That helped confirm that the services were working and that the verification logic was the problem.

For the model-result problems, I read the returned answers instead of checking only whether the cache reported HIT or MISS. The “500 to 1,500 words” answer disagreed with the 2000-character field limit in my code. For the false hits, the incoming question referred to another trial (CT-2026-102, later CT-2026-103), but the cached answer clearly referred to CT-2026-101.

4. What changed and why it works now

I reran the Part 1 check using id 5010, and it correctly showed 25 views in Redis and 20 persisted in the database. I also changed the verification script so it finds the trial using its trial code instead of assuming a fixed database id. The verification now passes.

I did not hide the two model-result problems. The detailed false-hit example in Part 3 is the CT-2026-103 question, which matched the cached CT-2026-101 question at similarity 0.9654 and returned a capacity of 80 instead of 60. In my threshold sweep, none of the tested thresholds from 0.75 to 0.95 completely

removed false hits. The version-marker check in Part 3.5 prevents stale answers after the underlying database record changes, but it does not prevent a semantically similar question about the wrong trial from matching the cache.

A wrong description-length answer is also used in Part 4. Without lessons, the agent answered "1,000", the Evaluator marked it REVISE, and the reusable lesson stores the correct 2000-character limit, so the same question passed once the lesson was loaded.

I also note three limitations in the report. First, the Evaluator is rule-based and can only judge questions covered by my reference dataset. Second, the repair workflow reaches 16/16 partly because the feedback includes the reference value, which makes correction easier for the Answering Agent. Third, on my 16 test questions the validated lessons had the same final accuracy as using all lessons. The validation step was still useful because it rejected the deliberately bad lesson, but it did not improve the final score on this small test set.

Appendix: cache_cases.yaml

cache_cases.yaml (committed before the first sweep)

```
seed_queries:
  - {id: S01, query: "What is the participant capacity of trial CT-2026-101?"}
  - {id: S02, query: "What is the title of trial CT-2026-103?"}
  - {id: S03, query: "What is the participant capacity of trial CT-2026-105?"}
  - {id: S04, query: "Which fields are required when creating a trial listing?"}
  - {id: S05, query: "How long can a trial description be?"}
  - {id: S06, query: "How long can a trial title be?"}
  - {id: S07, query: "What is the minimum participant capacity for a trial?"}
  - {id: S08, query: "How long can a trial code be?"}
  - {id: S09, query: "Which operations does the trial worker support?"}
  - {id: S10, query: "How many attempts does an agent make before a message goes to the dead-
letter topic?"}

cases:
  # ---- 10 exact duplicates (should hit) ----
  - {id: D01, type: exact_duplicate, seed: S01, expected_hit: true, query: "What is the
participant capacity of trial CT-2026-101?"}
  - {id: D02, type: exact_duplicate, seed: S02, expected_hit: true, query: "What is the title
of trial CT-2026-103?"}
  - {id: D03, type: exact_duplicate, seed: S03, expected_hit: true, query: "What is the
participant capacity of trial CT-2026-105?"}
  - {id: D04, type: exact_duplicate, seed: S04, expected_hit: true, query: "Which fields are
required when creating a trial listing?"}
  - {id: D05, type: exact_duplicate, seed: S05, expected_hit: true, query: "How long can a
trial description be?"}
  - {id: D06, type: exact_duplicate, seed: S06, expected_hit: true, query: "How long can a
trial title be?"}
  - {id: D07, type: exact_duplicate, seed: S07, expected_hit: true, query: "What is the
minimum participant capacity for a trial?"}
  - {id: D08, type: exact_duplicate, seed: S08, expected_hit: true, query: "How long can a
trial code be?"}
  - {id: D09, type: exact_duplicate, seed: S09, expected_hit: true, query: "Which operations
does the trial worker support?"}
  - {id: D10, type: exact_duplicate, seed: S10, expected_hit: true, query: "How many attempts
does an agent make before a message goes to the dead-letter topic?"}

  # ---- 10 paraphrases that should hit (same meaning, different words) ----
  - {id: P01, type: paraphrase, seed: S01, expected_hit: true, query: "How many participants
can enroll in clinical trial CT-2026-101?"}
  - {id: P02, type: paraphrase, seed: S02, expected_hit: true, query: "What is trial CT-2026-
103 called?"}
  - {id: P03, type: paraphrase, seed: S03, expected_hit: true, query: "What is the maximum
number of participants for trial CT-2026-105?"}
  - {id: P04, type: paraphrase, seed: S04, expected_hit: true, query: "What do I have to fill
in to create a new trial listing?"}
  - {id: P05, type: paraphrase, seed: S05, expected_hit: true, query: "What is the maximum
length of the description of a trial?"}
  - {id: P06, type: paraphrase, seed: S06, expected_hit: true, query: "What is the maximum
number of characters in a trial title?"}
  - {id: P07, type: paraphrase, seed: S07, expected_hit: true, query: "What is the smallest
participant capacity a trial may have?"}
  - {id: P08, type: paraphrase, seed: S08, expected_hit: true, query: "What is the maximum
length of a trial code?"}
  - {id: P09, type: paraphrase, seed: S09, expected_hit: true, query: "What requests can the
entity worker handle?"}
```

```
  - {id: P10, type: paraphrase, seed: S10, expected_hit: true, query: "After how many failed
tries is a message sent to the dead-letter topic?"}

  # ---- 10 superficially similar, different correct answer (should miss) ----
  - {id: X01, type: similar_different, seed: S01, expected_hit: false, query: "What is the
participant capacity of trial CT-2026-102?"}
  - {id: X02, type: similar_different, seed: S02, expected_hit: false, query: "What is the
title of trial CT-2026-104?"}
  - {id: X03, type: similar_different, seed: S03, expected_hit: false, query: "What is the
participant capacity of trial CT-2026-103?"}
  - {id: X04, type: similar_different, seed: S04, expected_hit: false, query: "Which fields
are optional when creating a trial listing?"}
  - {id: X05, type: similar_different, seed: S05, expected_hit: false, query: "What is the
minimum length of a trial description?"}
  - {id: X06, type: similar_different, seed: S06, expected_hit: false, query: "How long can a
trial category be?"}
  - {id: X07, type: similar_different, seed: S07, expected_hit: false, query: "What is the
maximum participant capacity for a trial?"}
  - {id: X08, type: similar_different, seed: S08, expected_hit: false, query: "How long can a
submitter email address be?"}
  - {id: X09, type: similar_different, seed: S09, expected_hit: false, query: "Which topic
does the worker send its responses to?"}
  - {id: X10, type: similar_different, seed: S10, expected_hit: false, query: "How many
seconds does an agent wait before the second attempt?"}

  # ---- 10 unrelated questions (should miss) ----
  - {id: U01, type: unrelated, seed: null, expected_hit: false, query: "What is the capital of
France?"}
  - {id: U02, type: unrelated, seed: null, expected_hit: false, query: "How do I bake
sourdough bread?"}
  - {id: U03, type: unrelated, seed: null, expected_hit: false, query: "Who won the 2018
football World Cup?"}
  - {id: U04, type: unrelated, seed: null, expected_hit: false, query: "Explain how
photosynthesis works."}
  - {id: U05, type: unrelated, seed: null, expected_hit: false, query: "What is the boiling
point of water in Fahrenheit?"}
  - {id: U06, type: unrelated, seed: null, expected_hit: false, query: "Recommend a good
laptop for programming."}
  - {id: U07, type: unrelated, seed: null, expected_hit: false, query: "How do I change a flat
tire?"}
  - {id: U08, type: unrelated, seed: null, expected_hit: false, query: "What is the plot of
Hamlet?"}
  - {id: U09, type: unrelated, seed: null, expected_hit: false, query: "Convert 10 miles to
kilometers."}
  - {id: U10, type: unrelated, seed: null, expected_hit: false, query: "Who painted the Mona
Lisa?"}
```